ScopeFlow — Frontend & UI Design Guide

Version 0.1 · 13 Oct 2026 · React + shadcn/ui · Companion to the Backend Design doc

1. Design philosophy

ScopeFlow is a tool people use when **money and trust are at stake**. The interface should feel calm, precise and honest.

1. **Evidence over decoration.** Every screen answers one question: *what was agreed, what is the evidence, what happens next?* If an element doesn't help answer it, remove it.
2. **Quiet by default.** Neutral surfaces, one accent colour, generous whitespace. Colour is reserved for meaning (status, risk, action).
3. **Never hide uncertainty.** "Unverified" and "Needs human" look as deliberate as "Met". The UI must never imply a pass that evidence doesn't support.
4. **Motion explains, never entertains.** Transitions show where something came from or went; they are short, consistent and respect reduced-motion settings.
5. **Irreversible actions feel heavy.** Approve, dispute and pay are slower, clearer and confirmed. Reading and browsing are fast and light.
6. **Build once, reuse everywhere.** Screens are assembled from a small set of decoupled components; no screen owns bespoke UI.

2. Stack

Concern	Choice	Notes
Framework	React 18+ with Vite (or Next.js if you want SSR)	A separate Node API exists, so a Vite single-page app is the simplest fit
Components	**shadcn/ui** everywhere	Copied into your repo, built on Radix primitives; you own the code
Styling	Tailwind CSS + CSS variables for tokens	shadcn's native approach; makes theming and dark mode trivial
Variants	class-variance-authority (cva) + `cn()` helper	Standard shadcn pattern for component variants
Motion	Motion (the library formerly called Framer Motion)	Layout, presence and shared-element transitions

Concern	Choice	Notes
Carousels	shadcn `Carousel` (built on Embla)	Swipe, keyboard and plugin support
Loading orbs	`thinking-orbs` (wrapped, see §8)	Canvas-based agent status indicator
Server state	TanStack Query	Caching, retries, polling for verification status
Forms	React Hook Form + Zod	Same validation vocabulary as the API edge
Routing	React Router (or TanStack Router)	Route-level code splitting
Client state	Local component state first; small Zustand store only for UI-wide concerns	Avoid global state for server data
Icons	lucide-react	Ships with shadcn
Docs & isolation	Storybook	Develop every shared component without the backend
Mocking	MSW (Mock Service Worker)	Build the UI in parallel with the backend

3. Visual language

3.1 Tokens (defined once as CSS variables)

- **Neutrals:** a single grey scale for background, surface, border, muted text and text. Light and dark themes both defined from day one.
- **Accent:** one hue for primary actions and focus rings.
- **Semantic status colours** used only for evidence and risk:
  - Met (green), Not met (red), Unverified (amber), Needs human (blue/violet), Not applicable (grey).
  - Always paired with an icon and a text label — never colour alone.
- **Typography:** one sans-serif family (Inter or Geist) plus a monospace for hashes, SHAs and logs. Three or four sizes in practice.
- **Spacing:** a 4px base grid; most layouts use 8/12/16/24/32.
- **Radius and elevation:** one radius for controls, one for cards; borders preferred over shadows.

3.2 Motion tokens

Token	Value	Use
`--motion-fast`	120 ms	Hover, press, toggles
`--motion-base`	200 ms	Dialogs, tabs, list changes
`--motion-slow`	320 ms	Page transitions, carousel snap, stepper progress
`--ease-out`	ease-out curve	Elements entering
`--ease-in-out`	symmetric curve	Elements moving across the screen

All durations come from tokens, so the whole product can be retuned in one place.

4. Component architecture (decoupling rules)

4.1 Four layers, one direction of dependency

```
pages/        route-level screens: compose features, no styling logic
   ↓
features/     milestone, evidence, spec, payments: know the domain and the
API
   ↓
patterns/     reusable composites: StatusBadge, EvidenceCard, AsyncBoundary
   ↓
ui/           shadcn primitives, kept close to upstream (Button, Dialog,
Carousel...)
```

- **`ui/`** is shadcn code. Change it only through tokens and variants, so upstream updates remain easy to merge.
- **`patterns/`** are domain-agnostic building blocks. They take plain props and emit callbacks. They never fetch data, never import from `features/`, and never read global state.
- **`features/`** own data fetching, API types and business wording. They compose patterns.
- **`pages/`** only arrange features and handle routing.

Enforce the direction with ESLint import-boundary rules so a shared component can never depend on a feature.

4.2 Rules every reusable component follows

1. **Presentational vs container:** presentational components receive data and callbacks; containers (hooks or feature components) fetch and mutate.
2. **Props down, events up.** No component reaches into context or a store for something it could receive as a prop.
3. **Compound components** for anything with parts (e.g. `Evidence.Root / .Header / .Body / .Actions`) instead of long prop lists.
4. **Variants via cva**, never boolean-prop sprawl.
5. **Headless logic in hooks** (`useStepper`, `useCountdown`, `useCarouselIndex`) so behaviour can be reused with different visuals.
6. **Slots over configuration:** accept `children` or render props for custom regions.
7. **Accessible by default:** keyboard, focus states, ARIA come from Radix/shadcn; custom parts must match.
8. **Every shared component has a Storybook story** covering default, loading, empty, error and long-content states.
9. **Domain types are mapped at the edge:** an API client layer converts server DTOs into UI view models, so components don't break when the backend payload changes.

4.3 Folder structure

```
src/
  ui/            shadcn primitives (generated)
  patterns/      StatusBadge, HashChip, MoneyText, AsyncBoundary, EmptyState,
                 ConfirmDialog, AgentStatus, PageTransition, CarouselShell...
  features/
    auth/ projects/ milestones/ spec/ evidence/ review/ payments/ audit/
      components/  hooks/  api/  model/
  pages/         Dashboard, Project, Milestone, Review, Payments, Settings
  lib/           cn(), formatters, motion variants, query client
  styles/        tokens.css, globals.css
```

5. Reusable component catalogue

Component	Layer	Purpose
`StatusBadge`	patterns	Met / Not met / Unverified / Needs human with icon + label, one source of truth for status styling

Component	Layer	Purpose
`MilestoneStepper`	patterns	Horizontal or vertical progress through the milestone state machine; current step animated
`CriterionRow` / `CriterionCard`	features/spec	One acceptance criterion with method, severity, blocking flag and status
`EvidenceCard`	patterns	Title, result, artifact link, hash, timestamp; slot for screenshot or log
`EvidenceCarousel`	patterns	Carousel of evidence items (screenshots, traces, test summaries)
`SpecDiff`	features/spec	Side-by-side comparison of two spec versions
`ApprovalBar`	patterns	Sticky action bar: Approve / Request changes / Dispute with confirmation
`MoneyText`	patterns	Locale-aware currency from integer minor units
`HashChip`	patterns	Monospace short hash or commit SHA with copy button and tooltip
`ActivityTimeline`	patterns	Ordered, append-only event list used for audit and history
`AsyncBoundary`	patterns	One wrapper that renders loading, error, empty or content consistently
`EmptyState`	patterns	Illustration-free message with a single next action
`ConfirmDialog`	patterns	Reusable confirmation for irreversible actions, with typed confirmation for payments
`AgentStatus`	patterns	Wrapper around the thinking orb plus label (see §8)
`PageTransition`	patterns	Route-level enter/exit animation wrapper

6. Transitions and motion

- **Page transitions:** short fade plus a small vertical shift (about 8px) on route change. Wrap routes once in `PageTransition`; individual pages stay unaware.
- **Shared-element motion:** when a milestone card opens into its detail view, animate the card's title and amount so the user keeps context.
- **Lists:** items entering or leaving animate height and opacity; reordering uses layout animation.
- **Status changes:** when a criterion changes from Unverified to Met, crossfade the badge and briefly highlight the row so the change is noticed.
- **Stepper:** progress line fills with the slow duration; completed steps get a subtle check animation.
- **Dialogs and sheets:** shadcn defaults, tuned to motion tokens.
- **Payment and approval:** no flashy celebration; a calm confirmation state with receipt details.
- **Reduced motion:** a global hook reads `prefers-reduced-motion` and the motion variants collapse to instant or opacity-only changes.
- **Budget:** one animated thing per interaction; nothing loops indefinitely except an active loading indicator.

7. Carousels

Use the shadcn `Carousel` wrapped in one `CarouselShell` pattern so behaviour is identical everywhere.

Where	What slides	Notes
Evidence review	Screenshots and browser-trace frames per criterion	Arrow keys, dot indicators, caption with timestamp and run id
Dashboard	Active milestone cards on narrow screens	Peek of next card to signal more
Spec history	Spec versions as cards	Select two to open `SpecDiff`
First-run onboarding	Three short steps explaining the flow	Skippable; shown once

Guidelines: always provide keyboard and button navigation (not swipe only), show position ("3 of 7"), never auto-advance content that someone needs to read, and lazy-load slide images.

8. Loading screens and the thinking orb

8.1 What the library is

Several React packages use the name "thinking orbs". The one that best matches your request is `thinking-orbs`: dotted, canvas-rendered orbs for AI and agent status, with named states (working, searching, solving, listening, composing, shaping), two tuned sizes (64px for avatar-scale and 20px for inline), automatic dark/light behaviour, speed and pause controls, and a static frame under reduced-motion. It requires React 18 or newer and is listed under an MIT licence on the 21st.dev component directory. A different package, `@yogesharc/thinking-orbs`, offers 8 states, many variants and shapes at a small bundle size. **Confirm the licence, maintenance activity and bundle size of whichever you choose before depending on it.**

8.2 Wrap it, never scatter it

Create one `AgentStatus` pattern that is the only file importing the library. Everything else uses `<AgentStatus state="verifying" label="Running checks" />`. If you replace the library or the design changes, only that file changes. It maps ScopeFlow meanings to orb states:

ScopeFlow situation	Orb state
Waiting for CI results or a webhook	working
Fetching repository, artifacts or evidence	searching
Running checks / tests (and later mutation or browser audits)	solving
Generating the evidence report	composing
AI-drafted criteria (later weeks)	shaping
Payment awaiting provider confirmation	working, with a clear text label

The orb must disappear when work ends and be replaced by the real outcome (success, error, cancelled). It is never a permanent decoration.

8.3 Loading hierarchy

1. **Instant (under ~300 ms):** show nothing; avoid flicker.
2. **Content loading:** shadcn `Skeleton` shaped like the final layout.

3. **Long-running or agent work:** `AgentStatus` with a text label and, where known, elapsed time or step.
4. **App boot / full-page splash:** centred 64px orb with the product name, shown only on first load.

Always pair the orb with a text label for screen readers, and show a way to cancel or leave the page for operations longer than a few seconds.

9. Key screens

Screen	Layout idea
Dashboard	Project list with milestone cards showing `MilestoneStepper`, amount and the next required action; filter by role
Project	Header with parties and budget; milestones in a vertical list; activity timeline in a side panel
Milestone detail	Two-pane view: left, the agreed spec (versioned, with acceptance status); right, evidence per criterion; sticky `ApprovalBar`
Spec editor	Criteria as editable rows; clear "new version" banner; both parties' acceptance state shown
Review	Criterion list with evidence carousel; filters for Not met / Unverified; comments tied to criteria
Payments	Funding and release status, sandbox labelling, receipt, reconciliation state
Audit & export	Read-only timeline and evidence bundle download
Auth / invite	Single centred card; invite links land on a clear "you've been invited to…" screen

Responsive rule: clients often approve from phones, so the review and approval flow must be fully usable on a narrow screen (single column, sticky action bar, carousels for evidence).

10. Data and state patterns

- **API client layer** (`features/*/api`) returns typed view models; components never see raw responses.

- **TanStack Query** for all server data; poll or refetch verification and payment status while a job is active, then stop.
- **Optimistic updates** for harmless actions (comments); never for approvals or payments — wait for the server and show the pending state.
- **Forms:** React Hook Form + Zod schemas that mirror server validation messages.
- **Errors:** one error presentation pattern (inline for fields, toast for transient, `AsyncBoundary` for page failures).

11. Accessibility and quality bar

- Keyboard reachable everywhere; visible focus ring using the accent token.
- Status always uses icon plus text plus colour.
- Contrast checked in light and dark themes.
- Respect reduced-motion and avoid animations that can't be paused.
- Test with a screen reader on the review and approval flow.
- Storybook accessibility checks and Playwright smoke tests in CI.

12. Delivery plan

The backend occupies week 1, so UI work runs against mocks.

Phase	Focus	Done when
UI-0 (1–2 days)	Vite app, Tailwind, shadcn init, tokens, light/dark, Storybook, MSW, lint boundaries	Theme switch and one story work
UI-1 (2–3 days)	Patterns: StatusBadge, MoneyText, HashChip, AsyncBoundary, EmptyState, ConfirmDialog, PageTransition, AgentStatus	Every pattern has stories for all states
UI-2 (3–4 days)	Auth, dashboard, project and milestone screens against MSW; stepper and spec views	Happy path navigable with mocked data
UI-3 (3–4 days)	Evidence review, carousel, ApprovalBar, payments screens; switch from MSW to the real API	Full flow works against the real backend
UI-4 (2 days)	Motion polish, responsive pass, accessibility pass, demo data	Demo script runs smoothly on desktop and phone

13. Open decisions

1. Vite SPA or Next.js (recommendation: Vite SPA).
2. Which thinking-orb package and licence check outcome.
3. Font choice (Inter versus Geist) and the single accent hue.
4. Whether to start a Storybook-first workflow now or add Storybook after the first screens.
5. English-only versus multi-language from the start.